

Guía de Estudio Completa

Proyecto #7: Redes Neuronales — Entrenamiento paralelo de MLP

Computación Paralela y Distribuida (CS4052) – 2026-I

Todo explicado desde cero, sin asumir conocimientos previos

Kalos Lazo Mera Aaron C. A. Navarro Mendoza Fernando A. Usurin Arias

Julio de 2026

Índice

PARTE I — Fundamentos de computación paralela	5
1. ¿Qué es la computación paralela?	5
2. Procesador, núcleo, hilo, proceso	5
3. Memoria compartida vs memoria distribuida	6
3.1. Memoria compartida (shared memory)	6
3.2. Memoria distribuida (distributed memory)	6
4. Tiempo secuencial y tiempo paralelo	6
5. Speedup (aceleración)	7
6. Eficiencia	7
7. La ley de Amdahl	7
8. La ley de Gustafson (weak scaling)	8
9. Strong scaling vs weak scaling	8
10. FLOPs y GFLOP/s	8
PARTE II — El modelo PRAM	9
11. ¿Qué es PRAM?	9
12. Los 4 sabores de PRAM (EREW, CREW, ERCW, CRCW)	9
13. El paradigma Work–Time (WT)	10

14. La palabra clave <code>pardo</code>	10
15. $\text{Work } W$ y $\text{Span } T_\infty$ con detalle	10
15.1. $\text{Work } W$	10
15.2. $\text{Span } T_\infty$	10
16. El Teorema de Brent	11
17. Costo C_p y WT-optimalidad	11
PARTE III — Redes neuronales (MLP) desde cero	11
18. ¿Qué es una red neuronal?	11
19. La neurona artificial	11
20. MLP: la estructura	12
21. Forward propagation (propagación hacia adelante)	12
22. Función de activación	13
23. Función de pérdida (loss)	13
24. Backpropagation (retropropagación)	13
25. Descenso de gradiente	13
26. Épocas, learning rate, batch	13
27. Accuracy (exactitud)	14
28. Los hiperparámetros del proyecto	14
29. Las semillas y por qué entrenar varios MLP	14
PARTE IV — El proyecto	14
30. El enunciado (proyecto #7)	14
31. El algoritmo secuencial (Algoritmo 1)	15
32. ¿Por qué es “embarrassingly parallel”?	15
PARTE V — El algoritmo paralelo	15
33. La idea del algoritmo paralelo	15

34.El modelo PRAM CREW (corregido)	16
35.Las correcciones que hicimos al parcial	16
36.Justificación de la memoria compartida (apregunta favorita del profe!)	16
37.OpenMP, multiprocessing y el GIL	17
38.Fork-join	17
39.fork vs spawn (detalle del código)	17
PARTE VI — Las métricas, paso a paso	17
40.Setup numérico	17
41.Work W (trabajo total)	18
42.Span T_∞	18
43.Tiempo paralelo T_p	18
44.Costo C_p y WT-optimalidad	18
45.Speedup y eficiencia (ejemplo real, Khipu)	18
46.Cálculo de FLOPs	19
PARTE VII — La implementación	19
47.Las 3 versiones beta	19
48.Worker crítico (cómo se mide el tiempo paralelo)	19
49.Cómo se descompone el tiempo	19
50.Reparto balanceado de semillas	20
PARTE VIII — Los resultados, interpretados	20
51.El hallazgo más importante: dos plataformas	20
52.¿Por qué el speedup no llega a los núcleos totales?	20
53.Validación con la teoría (el ajuste, Khipu)	21
54.¿ E se mantiene constante? (pregunta del enunciado)	21

PARTE IX — La exposición oral	21
55.Estructura recomendada (10 minutos)	21
56.Preguntas que hará el profe (con respuestas)	21
57.Trampas que caza el profe	22
PARTE X — Chuleta final	23
58.Diccionario rápido	23
59.Las fórmulas que debes saber	23
60.Los números del proyecto (memorízalos)	23

Cómo usar esta guía

Este documento explica **absolutamente todo** lo que necesitas saber para entender el proyecto y defender la exposición oral. Está ordenado de lo más básico a lo específico. Si lees de principio a fin, entenderás:

1. Qué es la computación paralela y sus conceptos (Parte I).
2. El modelo teórico PRAM que usa el curso (Parte II).
3. Qué es una red neuronal / MLP (Parte III).
4. El problema concreto del proyecto (Parte IV).
5. El algoritmo paralelo que diseñamos (Parte V).
6. Cómo se calculan las métricas, con números reales (Parte VI).
7. Cómo está hecho el código (Parte VII).
8. Qué significan los resultados (Parte VIII).
9. Cómo exponer y qué preguntará el profe (Parte IX).

Las cajas **naranjas** marcan lo MÁS IMPORTANTE.

PARTE I — Fundamentos de computación paralela

1. ¿Qué es la computación paralela?

Un programa **secuencial** ejecuta una instrucción tras otra, en un solo procesador, una a la vez. Si una tarea tarda 10 horas, así se queda.

La **computación paralela** consiste en usar **varios procesadores a la vez** para resolver un problema **más rápido**. La idea es dividir el trabajo en partes independientes y que cada procesador haga una parte al mismo tiempo (*en paralelo*).

Ejemplo cotidiano. Si tienes que lavar 100 platos y lo haces solo, tardas 100 minutos. Si invitas a 4 amigos y cada uno lava 25 platos, terminan en ≈ 25 minutos. Eso es paralelismo: el trabajo total es el mismo, pero lo repartiste.

Idea clave

Paralelizar NO reduce el trabajo total (los 100 platos se lavan igual). Reduce el **tiempo de reloj** (cuánto tardas tú en ver el resultado) porque varias unidades de trabajo ocurren *al mismo tiempo*.

2. Procesador, núcleo, hilo, proceso

- **CPU / procesador**: el “cerebro” del computador que ejecuta instrucciones.
- **Núcleo (core)**: una unidad de cálculo dentro de la CPU. Una CPU moderna tiene **varios**

núcleos (tu MacBook M5 tiene 10; un nodo del cluster Khipu tiene 32). Cada núcleo puede ejecutar una secuencia de instrucciones a la vez.

- **Hilo (thread)**: una secuencia de instrucciones en ejecución. Un núcleo puede cambiar rápido entre hilos, pero realmente hace uno a la vez (o dos con *hyper-threading*).
- **Proceso**: un programa en ejecución, con su propia memoria. Un proceso puede tener varios hilos dentro.

En este proyecto usamos **procesos** (no hilos) para paralelizar. Verás por qué en la Parte V (el GIL de Python).

3. Memoria compartida vs memoria distribuida

Esta es LA distinción más importante del curso. Hay dos formas de organizar una máquina paralela:

3.1. Memoria compartida (shared memory)

Todos los procesadores/núcleos ven **la misma memoria**. Si un procesador escribe un dato en la dirección 1000, los demás lo pueden leer de ahí inmediatamente. Comunicarse entre procesadores = leer/escribir variables compartidas. Rápido y simple.

Tecnologías: **OpenMP** (en C/C++/Fortran), **multithreading**, **multiprocessing** (Python). Tu CPU multinúcleo es de memoria compartida.

3.2. Memoria distribuida (distributed memory)

Cada procesador tiene **su propia memoria privada**. Para que el procesador B vea un dato del procesador A, A debe *enviarle un mensaje* (copiar los datos por la red). Comunicarse = enviar y recibir mensajes.

Tecnología: **MPI** (Message Passing Interface). Los clústers de muchas computadoras son de memoria distribuida.

Comparación rápida		
	Compartida	Distribuida
Memoria	Una sola, global	Una por procesador
Comunicación	Leer/escribir variables	Enviar mensajes (MPI)
Ejemplo	Tu laptop multinúcleo	Clúster de PCs
Librería típica	OpenMP	MPI
¿Copiar datos?	No, ya están	Sí, hay que enviarlos

En nuestro proyecto usamos memoria compartida. La justificación está en la Parte V (los datos son compartidos y de solo lectura, así que es más eficiente no tener que copiarlos).

4. Tiempo secuencial y tiempo paralelo

- T_s (tiempo secuencial): cuánto tarda el programa con **1** solo procesador. Es tu línea base.
- T_p (tiempo paralelo): cuánto tarda con p procesadores.

El objetivo siempre es que $T_p < T_s$, y cuanto menor, mejor.

5. Speedup (aceleración)

El **speedup** mide cuántas veces más rápido es el paralelo:

$$S(p) = \frac{T_s}{T_p}$$

Ejemplo con datos reales de nuestro proyecto (Khipu, n=40000 muestras):

- $T_s = T_1 = 305$ s (con 1 proceso).
- Con $p = 8$ procesos: $T_8 = 53$ s.
- $S(8) = 305/53 = 5,7$. O sea, 5.7 veces más rápido.

El **speedup ideal** es $S(p) = p$ (el doble de procesadores $\Rightarrow$ el doble de rápido). En la práctica casi nunca se alcanza.

6. Eficiencia

La **eficiencia** es el speedup “por procesador”:

$$E(p) = \frac{S(p)}{p} = \frac{T_s}{p \cdot T_p}$$

Mide qué *tan bien* estás aprovechando los procesadores:

- $E = 1$ (100 %): aprovechamiento perfecto.
- $E = 0,5$: estás usando la mitad del potencial.

Ejemplo real: con $p = 4$, $S(4) = 3,5$, entonces $E(4) = 3,5/4 = 0,87$. Estás aprovechando el 87 % de los 4 procesadores. ¡Bastante bueno!

7. La ley de Amdahl

Ley de Amdahl — la idea central

Ningún programa es 100 % paralelizable. Siempre hay una **fracción secuencial** f que no se puede paralelizar (leer datos, inicializar, reducir el resultado final...). Esa fracción fija un **techo** al speedup por muchos procesadores que agregues.

Si una fracción f del programa es secuencial y el resto $(1 - f)$ es paralela, entonces:

$$S(p) \leq \frac{1}{f + \frac{1-f}{p}}$$

Cuando $p \rightarrow \infty$ (procesadores infinitos), $S \rightarrow 1/f$. **El speedup máximo es $1/f$, por muchos núcleos que tengas.**

Ejemplo: si $f = 0,1$ (10 % secuencial), el speedup máximo es $1/0,1 = 10$, aunque tengas 1000 núcleos. Esa fracción secuencial te condena.

En nuestro proyecto, S satura en ≈ 5 en la Mac (10 núcleos) y $\approx 10,6$ en Khipu (32 núcleos). Para la Mac, eso implica una fracción efectiva $f \approx 1/5 = 0,2$. ¿De dónde viene ese 20 % “secuencial”? No es código secuencial puro, sino que los procesos se estorban (contención de memoria) y el hyper-threading infla el conteo de núcleos. Lo veremos en la Parte VIII.

8. La ley de Gustafson (weak scaling)

Amdahl asume que el tamaño del problema es fijo. Gustafson dice: ¿y si cuando agregas procesadores también agrandas el problema? (cada procesador hace el mismo trabajo pero en total se resuelve algo más grande).

Bajo este “weak scaling”, el speedup puede crecer *casi lineal* con p , porque la fracción secuencial deja de dominar.

Resumen Amdahl vs Gustafson		
	Amdahl	Gustafson
Tamaño del problema	Fijo	Crece con p
Tipo de escalabilidad	Strong scaling	Weak scaling
Speedup	Techo limitado por f	Casi lineal
Eficiencia E	Cae con p	Se mantiene ≈ 1

9. Strong scaling vs weak scaling

- **Strong scaling:** mantienes el problema **fijo** (mismos datos) y aumentas p . Pregunta: “¿termina más rápido?” Mide T_p vs p . Ideal: $T_p = T_s/p$.
- **Weak scaling:** aumentas el problema **junto con** p (cada procesador siempre hace el mismo trabajo). Pregunta: “¿sigue tardando lo mismo?” Mide $T_p(n \cdot p)/T_p(n)$; ideal = 1.

10. FLOPs y GFLOP/s

- **FLOP** = *F*loating-*p*oint *O*Peration: una operación de coma flotante (una suma o multiplicación con decimales).
- **FLOPs** (con minúscula): cantidad total de operaciones.
- **FLOP/s** o **GFLOP/s**: operaciones por segundo (giga = 10^9). Mide el **rendimiento**: cuán rápido calcula la máquina.

Fórmula: $\text{GFLOP/s} = \frac{\text{FLOPs totales}}{T_p \cdot 10^9}$.

A más GFLOP/s, más rápido calcula. En nuestro proyecto, el rendimiento pico es ≈ 53 GFLOP/s.

Las 3 métricas que pide el enunciado
1. Tiempo de ejecución T_p (segundos).
2. Speedup $S(p) = T_s/T_p$.
3. Eficiencia $E(p) = S(p)/p$.
Y de regalo: GFLOP/s (rendimiento).

PARTE II — El modelo PRAM

11. ¿Qué es PRAM?

PRAM = *Parallel Random Access Machine*. Es un **modelo teórico** (como una receta abstracta) para describir algoritmos paralelos. No es una máquina real; es una forma de **escribir y analizar** algoritmos paralelos de manera limpia.

Imagina una máquina con:

- Un número p de **procesadores** idénticos: $P_0, P_1, \dots, P_{p-1}$.
- Una **memoria global compartida** con celdas.
- Un **reloj común**: en cada “paso” de reloj, todos los procesadores ejecutan una instrucción a la vez.

El PRAM es la forma estándar del curso de **describir un algoritmo paralelo en pseudocódigo** y calcular su complejidad.

12. Los 4 sabores de PRAM (EREW, CREW, ERCW, CRCW)

El problema clave de la memoria compartida: **¿Qué pasa si dos procesadores quieren acceder a la MISMA celda de memoria al mismo tiempo?**

Depende de si es lectura o escritura:

- **Lectura concurrente**: ¿pueden varios leer la misma celda a la vez?
- **Escritura concurrente**: ¿pueden varios escribir en la misma celda a la vez?

De ahí salen 4 modelos (E=Exclusive, C=Concurrent):

Los 4 modelos PRAM		
Modelo	Lectura	Escritura
EREW	Exclusiva	Exclusiva
CREW	Concurrente	Exclusiva
ERCW	Exclusiva	Concurrente
CRCW	Concurrente	Concurrente

- **EREW** (el más estricto): ni leer ni escribir pueden coincidir. El más “caro” de simular pero el más realista en hardware básico.
- **CREW**: varios pueden LEER a la vez, pero si alguien ESCRIBE, debe ser el único. **Es el modelo que usamos en el proyecto.**
- **CRCW** (el más permisivo): hasta pueden escribir varios a la vez (hay que definir cómo se resuelve el conflicto). El más poderoso pero el menos realista.

¿Por qué elegimos CREW en el proyecto? Porque:

- Los p procesadores **leen el dataset concurrentemente** (lectura concurrente $\Rightarrow$ la C de CREW).

- Las escrituras del resultado son **exclusivas**: cada procesador escribe en su propia celda, sin pisarse (escritura exclusiva $\Rightarrow$ la EW de CREW).
- No necesitamos escritura concurrente (CRCW sería matar moscas a cañonazos).

13. El paradigma Work–Time (WT)

El curso usa una forma específica de escribir algoritmos PRAM llamada **Work–Time (WT)**. En vez de contar ciclos de reloj uno por uno, se escribe el pseudocódigo marcando qué partes van en paralelo, y luego se derivan dos cantidades:

- **Work W** : el trabajo total = número de operaciones = equivale al tiempo secuencial T_s .
- **Span T_∞** : el camino crítico = el tiempo con procesadores *infinitos*.

14. La palabra clave pardo

En el pseudocódigo WT, la palabra **pardo** (*parallel-do*) marca los bucles que se ejecutan **en paralelo**. Todo lo que está dentro de un **pardo** lo hacen todos los procesadores a la vez.

```
for k = 0 to p-1 pardo do
    <lo que hace cada procesador, en paralelo>
```

Corrección clave del parcial

En el informe parcial llamamos “**Broadcast**” al paso donde los procesadores leen los datos. **Eso estaba mal**: en memoria compartida NO existe un “broadcast”. Los datos ya están en memoria global; cada procesador simplemente los **lee concurrentemente** (eso es justamente el acceso CREW). Por eso en la versión final lo renombramos a “**lectura concurrente**” y lo marcamos con **pardo** donde corresponde. **No digas nunca “broadcast” en memoria compartida** o el profe te para.

15. Work W y Span T_∞ con detalle

15.1. Work W

Es el **total de operaciones** que hace el algoritmo, contando las de todos los procesadores. Si lo corriera un solo procesador secuencialmente, tardaría $O(W)$. Por eso W **equivale al tiempo secuencial T_s** .

15.2. Span T_∞

Es la duración del **camino crítico**: la cadena más larga de operaciones que dependen una de otra. Representa el tiempo que tardarías si tuvieras **procesadores infinitos** (por eso el subíndice ∞). Con procesadores infinitos, lo único que limita es qué operaciones **DEBEN** ir en secuencia.

Intuición: imagina una receta. Aunque tuvieras 1000 cocineros, no puedes servir el plato antes de hornearlo. El span es el tiempo mínimo irreducible (lo que forzosamente va en secuencia).

16. El Teorema de Brent

El Teorema de Brent relaciona W , T_∞ y el número real de procesadores p con el tiempo paralelo T_p :

$$T_p \leq \frac{W}{p} + T_\infty$$

Interpretación:

- $\frac{W}{p}$: si repartieras el trabajo W perfectamente entre p procesadores.
- T_∞ : el “mínimo irreducible” (camino crítico).
- El tiempo real está por debajo de la suma.

Esto da una cota **superior** del tiempo paralelo: en el peor caso, T_p no supera $W/p + T_\infty$.

17. Costo C_p y WT-optimalidad

Costo $C_p = T_p \cdot p$ (tiempo $\times$ número de procesadores). Es una medida del “esfuerzo total” gastado.

Un algoritmo PRAM es **WT-óptimo** si su costo es proporcional al trabajo:

$$C_p = O(W) \iff T_p = O(W/p)$$

Es decir, cuando el paralelo reparte *perfectamente* el trabajo (no desperdicia procesadores). Esto se cumple si $p \leq W/T_\infty$.

En nuestro proyecto

$W/T_\infty = O(p_s)$ (número de semillas). Como siempre usamos $p \leq p_s = 32$, el algoritmo es **WT-óptimo**. Bien.

PARTE III — Redes neuronales (MLP) desde cero

18. ¿Qué es una red neuronal?

Una **red neuronal artificial** es un modelo matemático inspirado (muy vagamente) en las neuronas del cerebro. Sirve para **aprender patrones a partir de datos**: le muestras muchos ejemplos y “aprende” a hacer una tarea (clasificar imágenes, predecir un número, etc.).

No es magia ni inteligencia: es un montón de **multiplicaciones y sumas** organizadas de forma ingeniosa, que se ajustan automáticamente para minimizar errores.

19. La neurona artificial

Una **neurona** (o “perceptrón”) es una función matemática simple:

$$y = f(w_1x_1 + w_2x_2 + \dots + w_dx_d + b)$$

- $x_1, \dots, x_d$: las **entradas** (los datos, ej. los píxeles de una imagen o las “features”).
- $w_1, \dots, w_d$: los **pesos** (números que la red aprende).
- b : el **sesgo** (bias), otro número que se aprende.
- f : la **función de activación** (veremos cuál).
- y : la **salida**.

La neurona calcula una **suma ponderada** de las entradas (cada entrada por su peso) más el sesgo, y le aplica una función.

20. MLP: la estructura

MLP = *Multi-Layer Perceptron* = perceptrón multicapa. Son varias neuronas organizadas en **capas**:

```
[ entrada ]  ->  [ capa oculta ]  ->  [ salida ]
   d neuronas      h neuronas          c neuronas
```

- **Capa de entrada**: recibe los datos. Tiene d valores (en nuestro proyecto $d = 32$ “features”).
- **Capa oculta**: las neuronas “intermedias” que hacen el trabajo pesado. Tiene h neuronas (en nuestro proyecto $h = 128$).
- **Capa de salida**: produce el resultado. Tiene c neuronas (en nuestro proyecto $c = 10$ clases).

Cada neurona de una capa está **conectada** a todas las de la siguiente, y cada conexión tiene un peso. De ahí sale el costo computacional (multiplicar matrices).

21. Forward propagation (propagación hacia adelante)

Es el cálculo de la salida a partir de la entrada. Paso a paso:

De la entrada a la capa oculta: cada neurona oculta j calcula:

$$z_j = \sum_{i=1}^d w_{ji}^{(1)} x_i + b_j^{(1)}, \quad a_j = f(z_j)$$

Son h neuronas, cada una hace d multiplicaciones. Total: $h \cdot d$ multiplicaciones. Con activación.

De la capa oculta a la salida: cada neurona de salida k calcula:

$$o_k = \sum_{j=1}^h w_{kj}^{(2)} a_j + b_k^{(2)}$$

Son c neuronas, cada una hace h multiplicaciones. Total: $c \cdot h$.

Total de operaciones por una muestra: $\approx 2 \cdot h \cdot d + 2 \cdot c \cdot h$ (contando multiplicaciones y sumas como FLOPs).

22. Función de activación

La función f introduce **no linealidad** (sin ella, una red de varias capas sería equivalente a una sola, y no podría aprender cosas complejas). Las típicas:

- **ReLU**: $f(z) = \max(0, z)$. Simple y muy usada.
- **Sigmoide**: aplasta a $[0, 1]$.
- **Softmax** (en la salida de clasificación): convierte las salidas en probabilidades (suman 1).

23. Función de pérdida (loss)

Después del forward, la red produce una predicción. La comparamos con la respuesta correcta y medimos el **error** con una función de pérdida. Para clasificación se usa típicamente la **entropía cruzada** (*cross-entropy*):

$$\text{Loss} = - \sum_k y_k \log(\hat{y}_k)$$

donde y es la respuesta correcta y $\hat{y}$ la predicción. Menor pérdida = mejor predicción.

24. Backpropagation (retropropagación)

Aquí está la magia del aprendizaje. La red quiere **reducir la pérdida**. Para eso necesita saber **cómo cambiar cada peso** para que el error baje.

Backpropagation calcula, usando la regla de la cadena del cálculo, el **gradiente** de la pérdida respecto a cada peso: $\partial \text{Loss} / \partial w$. Es decir, “cuánto contribuye cada peso al error”.

Costo: el backward cuesta más o menos lo mismo que el forward (hay que recorrer la red al revés). Por eso decimos que entrenar cuesta $\approx 2 \times$ un forward.

25. Descenso de gradiente

Una vez que sabes el gradiente, **ajustas los pesos** en dirección contraria al error:

$$w_{\text{nuevo}} = w_{\text{viejo}} - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

- η (**learning rate**, tasa de aprendizaje): qué tan grande es el paso. Si es muy grande, diverge; muy pequeño, aprende lento.

Repetido muchas veces, los pesos convergen a valores que minimizan el error.

26. Épocas, learning rate, batch

- **Época (epoch)**: una pasada completa por *todos* los datos de entrenamiento. En nuestro proyecto $E = 100$ épocas.
- **Batch**: en la práctica se procesan los datos en grupos (lotes) en vez de uno por uno. Nosotros usamos todo el conjunto.
- **Iteraciones**: el número de actualizaciones. Una época con n muestras y batch b tiene n/b iteraciones.

27. Accuracy (exactitud)

Métrica de calidad: fracción de predicciones correctas. Si de 100 ejemplos acierta 80, *accuracy* = 0,80. En nuestro proyecto los mejores modelos alcanzan $\approx 0,77$ – $0,87$ según el tamaño de los datos.

28. Los hiperparámetros del proyecto

Un **hiperparámetro** es una configuración que tú eliges (no la aprende la red). Los del proyecto:

- $h = 128$: neuronas de la capa oculta.
- $E = 100$ (`max_iter`): épocas máximas.
- $\alpha = 10^{-4}$: regularización L2 (evita sobreajuste).
- $\eta = 10^{-3}$ (`learning_rate_init`): learning rate.
- $d = 32$: features de entrada; $c = 10$: clases.

29. Las semillas y por qué entrenar varios MLP

La pregunta clave del proyecto

El entrenamiento de una red empieza con pesos **aleatorios**, generados por un **generador de números pseudoaleatorios** (RNG) que se controla con una **semilla** (*seed*). **Cambiar la semilla cambia el resultado**: distintos pesos iniciales $\Rightarrow$ distinto modelo $\Rightarrow$ distinta exactitud.

Como no sabemos de antemano qué semilla da el mejor modelo, una práctica robusta es **entrenar varios MLP con semillas distintas y conservar el de mayor exactitud**. **¡Eso es justo lo que paralelizamos!**

Entrenamos $p_s = 32$ MLP, cada uno con una semilla $s \in \{0, 1, \dots, 31\}$, y nos quedamos con el de mejor accuracy. Y como cada MLP es **independiente** de los demás, los podemos entrenar **en paralelo**.

PARTE IV — El proyecto

30. El enunciado (proyecto #7)

El proyecto pide: dado el pseudocódigo secuencial del Algoritmo 1 (entrenar varios MLP con distintas semillas y quedarse con el mejor), **paralelizarlo**.

Requisitos del enunciado:

1. Hacer un **PRAM** y el código en paralelo.
2. Usar $p \in \{1, 2, 4, 8, 16, 32\}$ procesos y $n \in \{5000, 10000, 20000, 40000\}$ muestras; generar gráficas de tiempos.
3. Registrar el desarrollo en **al menos 3 versiones beta**.

4. Comparar los resultados con la **complejidad teórica**, superponiendo la curva teórica a la experimental.
5. Calcular **speedup y eficiencia**, analizar la escalabilidad y responder: ¿se mantiene E constante?

31. El algoritmo secuencial (Algoritmo 1)

Algoritmo 1 — secuencial

```

Dividir (X, y) en train y test  (una sola vez)
mejor_accuracy = -inf
para cada semilla s en {0,...,31}:
    crear modelo M con semilla s
    entrenar M con (X_train, y_train)    <- esto es lo pesado
    A = accuracy(M, X_test, y_test)
    si A > mejor_accuracy:
        mejor_accuracy = A
        mejor_modelo = M
devolver mejor_modelo

```

El bucle entrena 32 modelos **uno detrás de otro**. Lento. Ese es nuestro T_s .

32. ¿Por qué es “embarrassingly parallel”?

Embarrassingly parallel = vergonzosamente paralelo

Es un término técnico (en serio) para problemas que se paralelizan de forma **trivial**: el trabajo se divide en partes **totalmente independientes**, sin necesidad de comunicación entre ellas durante el cómputo.

El entrenamiento de los 32 MLP es así: **cada MLP no necesita nada de los demás**. Solo se juntan al final para comparar accuracies y elegir el mejor (una reducción *arg-máx*). Por eso paraleliza genial.

PARTE V — El algoritmo paralelo

33. La idea del algoritmo paralelo

En vez de entrenar los 32 MLP uno por uno, los **repartimos** entre p procesadores. Cada procesador entrena $\lceil 32/p \rceil$ MLP y reporta su mejor modelo local. Al final, los p ganadores locales se comparan para sacar el global (reducción *arg-máx* en árbol).

34. El modelo PRAM CREW (corregido)

Algoritmo 2 — PRAM CREW (memoria compartida)

```
Paso 1 (secuencial, raíz):  
  Dividir (X,y) en train/test  
  Dejar los datos en MEMORIA COMPARTIDA  
  
Paso 2 (PARALELO, pardo):  <- lo que antes llamábamos "Broadcast"  
  para k=0..p-1 pardo:  
    P_k LEE de memoria compartida (X_train, ...) [lectura CREW]  
    P_k entrena sus semillas S_k y guarda su mejor local  
  
Pasos 3..(2+log2 p) (PARALELO, pardo): reducción arg-máx en árbol  
  para stride=1..log2(p):  
    para k pardo:  
      si mejor[k+stride] > mejor[k]: mejor[k] = mejor[k+stride]  
  
Devolución: mejor[0] = M*
```

35. Las correcciones que hicimos al parcial

1. **Quitar “Broadcast”.** En memoria compartida los datos ya están en memoria global; los procesadores los **leen concurrentemente** (acceso CREW). No hay nada que “difundir”.
2. **No paralelizar la lectura como paso aparte.** Es solo lectura de referencias ya compartidas.
3. **No medir la lectura ni el split en los tiempos.** Medimos solo la región paralelizable (entrenamiento) + la reducción.
4. **Justificar el uso de memoria compartida** (sección siguiente).
5. **Incluir el término $\log_2 p$** en la fórmula de T_p (viene de la reducción en árbol).

36. Justificación de la memoria compartida (apregunta favorita del profe!)

¿Por qué memoria compartida y no MPI?

1. **La comunicación es mínima.** Cada MLP es independiente; la única interacción es la reducción final (un árbol de $\log_2 p$ rondas). No se aprovecha nada del paso de mensajes.
2. **Los datos son compartidos y de solo lectura.** Cada MLP se entrena con *todo* el dataset. En memoria compartida, los p procesadores simplemente **leen** las mismas referencias (sin copiar). En MPI habría que **copiar el dataset a cada proceso** (memoria $\times p$ + tiempo de envío).
3. **Conclusión:** la memoria compartida es aquí **más eficiente**. MPI no aportaría nada y sumaría overhead.

37. OpenMP, multiprocessing y el GIL

- **OpenMP**: la forma estándar de paralelismo en memoria compartida en C/C++/Fortran. Usas directivas (`#pragma omp parallel`) para decir “este bucle va en paralelo”. Modelo *fork-join*.
- **GIL** (Global Interpreter Lock): un candado del intérprete de Python que impide que **dos hilos** ejecuten código Python a la vez. Por eso en Python los **hilos** no dan paralelismo real.
- **multiprocessing**: la solución de Python: en vez de hilos, usa **procesos** separados (cada uno con su intérprete y su GIL). Así sí hay paralelismo real. Es el análogo de OpenMP en Python.

En el proyecto: el MODELO conceptual es OpenMP (memoria compartida, fork-join), pero la IMPLEMENTACIÓN es **multiprocessing** porque el GIL impide hilos reales. Conceptualmente es lo mismo: p trabajadores que comparten memoria y se sincronizan al final.

38. Fork-join

Es el patrón de ejecución:

1. Un proceso “raíz” está corriendo.
2. **Fork**: el raíz crea p procesos trabajadores (heredan la memoria por *copy-on-write*, sin copiar).
3. Los p trabajadores hacen su parte **en paralelo**.
4. **Join**: el raíz espera a que todos terminen y recoge los resultados.

Es exactamente lo que hace `#pragma omp parallel` en OpenMP y `multiprocessing.Pool` en Python.

39. fork vs spawn (detalle del código)

- **fork**: el proceso hijo es un **clon** del padre (hereda la memoria por copy-on-write, sin copiar datos). **Eficiente**. Es lo fiel al modelo de memoria compartida. Lo usamos en la beta 3.
- **spawn**: el proceso hijo arranca **desde cero** y hay que **serializar** (con *pickle*) y enviarle los datos. Más lento. La beta 2 lo usaba (y por eso era ineficiente).

PARTE VI — Las métricas, paso a paso (con números reales)

Aquí derivamos cada métrica del pseudocódigo, con un ejemplo numérico real.

40. Setup numérico

Plataforma: **cluster Khipu, nodo n003** (Intel Xeon Gold 6130, 32 núcleos). Tomemos $n = 40000$ muestras. Parámetros: $p_s = 32$ semillas, $E = 100$ épocas, $d = 32$ features, $h = 128$ neuronas, $c = 10$ clases.

41. Work W (trabajo total)

El paso dominante es entrenar los 32 MLP. Cada MLP cuesta $O(E \cdot n \cdot d \cdot h)$:

$$W = O(p_s \cdot E \cdot n \cdot d \cdot h) = O(32 \cdot 100 \cdot 40000 \cdot 32 \cdot 128).$$

Esto **coincide con** T_s , el tiempo secuencial. Medido en Khipu: $T_s = 307$ s.

42. Span T_∞

Con procesadores infinitos, cada semilla tiene su propio procesador ($p = p_s = 32$), así que cada uno entrena un solo MLP:

$$T_\infty = O(E \cdot n \cdot d \cdot h) + O(\log p_s) = O(E \cdot n \cdot d \cdot h).$$

Es el tiempo de **un** MLP más la reducción. Irreducible.

43. Tiempo paralelo T_p

Por el Teorema de Brent, $T_p \leq W/p + T_\infty$. Como E, d, h son constantes en nuestros experimentos, se absorben en una constante a [s/muestra]. El modelo numérico que ajustamos a los datos es:

$$T_p \approx a \cdot \frac{p_s}{p} \cdot n + b \cdot \log_2 p$$

Ejemplo ($n = 40000$, $p = 2$):

$$T_2 \approx 2,42 \times 10^{-4} \cdot \frac{32}{2} \cdot 40000 + 2,18 \cdot \log_2 2 = 154,9 + 2,18 = 157 \text{ s.}$$

El medido fue 153,6 s. ¡Coincide! (la pequeñísima diferencia es ruido).

44. Costo C_p y WT-optimalidad

$C_p = T_p \cdot p$. Como $p \leq p_s = 32 = W/T_\infty$, el algoritmo es WT-óptimo: $C_p = O(W)$.

45. Speedup y eficiencia (ejemplo real, Khipu)

Para $n = 40000$ en Khipu (32 núcleos):

p	T_p (s)	Speedup S	Eficiencia E	GFLOP/s
1	305.3	1.00	1.00	7.2
2	153.6	1.99	0.99	14.3
4	108.9	2.80	0.70	20.3
8	53.2	5.73	0.72	41.4
16	33.0	9.26	0.58	66.8
32	28.9	10.58	0.33	76.3

Qué leer aquí:

- $p = 2$ es casi ideal (eficiencia 0.99).
- El speedup sigue creciendo hasta $p = 32$, donde alcanza $\sim 10,6$.
- La eficiencia cae a p alto (a $p = 32$, 0.33) por la contención de memoria y el hyper-threading (32 lógicos $\approx$ 16 físicos).
- El GFLOP/s sube hasta ~ 76 .

46. Cálculo de FLOPs

Para UN MLP (forward + backward, E épocas, n muestras, arquitectura $d-h-c$):

$$\text{FLOPs por MLP} = 4 \cdot E \cdot h \cdot (d + c) \cdot n.$$

Ejemplo ($n = 40000$): $4 \cdot 100 \cdot 128 \cdot (32 + 10) \cdot 40000 = 8,6 \times 10^{10}$ FLOPs = 86 GFLOPs por MLP. Los 32 MLP suman $\approx 2,75 \times 10^{12}$ FLOPs. A $T_8 = 53,2$ s (Khipu), GFLOP/s = $2,75 \times 10^{12} / (53,2 \cdot 10^9) \approx 52$. ✓ (En $p = 32$, $T = 28,9$ s $\Rightarrow \approx 76$ GFLOP/s, el pico.)

PARTE VII — La implementación

47. Las 3 versiones beta

El enunciado pide registrar el desarrollo en ≥ 3 pasos. Nuestras 3 betas:

1. **Beta 1 (beta1_secuencial.py)**: el Algoritmo 1 puro, secuencial. Es la **referencia** (da T_s y verifica correctitud).
2. **Beta 2 (beta2_paralelo.py)**: primer intento paralelo con `spawn`. Funciona pero es **ineficiente** porque serializa los datos a cada worker.
3. **Beta 3 (beta3_paralelo_opt.py)**: la versión final, con `fork` (los workers heredan los datos por copy-on-write, sin copiar), reparto balanceado de semillas y medición fina de tiempos. **Esta es la que usamos para experimentar.**

48. Worker crítico (cómo se mide el tiempo paralelo)

El tiempo paralelo NO es la suma

En paralelo, los p trabajadores corren **a la vez**. El tiempo total de esa fase es el del **trabajador más lento** (el último en terminar), NO la suma de todos. Si sumaras los tiempos estarías midiendo el *trabajo* W , no el *tiempo* T_p .

Por eso medimos `T_compute = max(tiempos de los workers)`.

49. Cómo se descompone el tiempo

- `T_compute`: tiempo del worker crítico (la región paralela).
- `T_reduce`: overhead de recoger resultados + arg-máx (**medido** $< 1,5\%$).

- $T_{\text{total}} = T_{\text{compute}} + T_{\text{reduce}}$.
- **NO medimos** la lectura de datos ni el split (corrección del profe).

50. Reparto balanceado de semillas

Repartimos las 32 semillas en p grupos de tamaño $\lceil 32/p \rceil$ o $\lfloor 32/p \rfloor$, para que funcione **aunque** p **no divida a 32**. En la práctica usamos $p \in \{1, 2, 4, 8, 16, 32\}$ que sí dividen a 32, pero el código es robusto.

PARTE VIII — Los resultados, interpretados

51. El hallazgo más importante: dos plataformas

Mismo algoritmo, dos hardwares: el speedup escala

Corrimos el mismo código en dos plataformas de memoria compartida: **MacBook M5 (10 núcleos)** y **cluster Khipu, nodo n003 (32 núcleos, Xeon Gold 6130)**. Resultado: el speedup pasa de ~ 5 (Mac) a ~ 10.6 (Khipu) al aumentar los núcleos. Eso **valida experimentalmente** que el algoritmo escala con el hardware, como predice la teoría. Y en ambas plataformas el speedup no llega a p (los núcleos totales): investiguemos por qué.

52. ¿Por qué el speedup no llega a los núcleos totales?

Descartamos causas con el método científico:

Hipótesis 1: ¿Es el overhead de gestión de procesos (fork)? Medimos T_{reduce} : es $< 1,5\%$. **Descartado.**

Hipótesis 2: ¿Es que sklearn paraleliza cada MLP con BLAS y se estorban? Medimos un MLP aislado con y sin limitar hilos: idéntico (threadpoolctl no detecta hilos). Limitamos el BLAS a 1 hilo por proceso para que la única paralelización sea la nuestra. **Descartado.**

Hipótesis 3 (correcta): contención de memoria + hyper-threading. El entrenamiento del MLP es *memory-bound* (accede a $n \cdot d$ datos por época). A alto p , los procesos compiten por el ancho de banda de memoria y cada MLP se vuelve más lento. Además, en el Xeon los “32 núcleos” son en realidad **16 físicos con hyper-threading** (32 lógicos), así que el speedup real máximo está entre 10 y 16. La Mac, sin hyper-threading, satura antes (~ 5 con sus 10 núcleos).

La lección clave

El speedup crece con el hardware (Mac $\sim 5 \rightarrow$ Khipu ~ 10.6), pero nunca llega a p porque (1) el hyper-threading infla el conteo de núcleos y (2) la memoria compartida se satura. Un análisis honesto que mide, descarta hipótesis y concluye con causas verificadas vale más que un speedup “perfecto”.

53. Validación con la teoría (el ajuste, Khipu)

Ajustamos el modelo $T_p = a \cdot (p_s/p) \cdot n + b \cdot \log_2 p$ por mínimos cuadrados no-negativos sobre los 48 puntos de Khipu:

$$a = 2,42 \times 10^{-4} \text{ s/muestra}, \quad b = 2,18 \text{ s}, \quad R^2 = 0,989.$$

$R^2 = 0,989$ significa que el modelo explica el 98.9 % de la variación de los datos. Un ajuste excelente. La teoría (con el término $\log_2 p$ de la reducción, que corregimos respecto al parcial) reproduce los datos. (En la Mac el ajuste dio $R^2 = 0,982$; ambos excelentes.)

Detalle de rigor: el modelo ajusta bien en la región lineal ($p \leq 4$); para $p > 8$ el experimental queda por encima de la teoría *justo por la contención de memoria* que no captura la fórmula lineal.

54. $\mathcal{E}$ se mantiene constante? (pregunta del enunciado)

Respuesta matizada:

- **Strong scaling** (Amdahl): $\mathcal{E}$ **NO** se mantiene; cae con p (por overhead + contención).
- **Weak scaling** (Gustafson): $\mathcal{E}$ **SÍ** se mantiene cercana a 1 (al crecer n con p , la carga por procesador es constante).

Lo mostramos en dos gráficas separadas.

PARTE IX — La exposición oral

55. Estructura recomendada (10 minutos)

1. **(1 min) Problema:** entrenar 32 MLP con semillas distintas, quedarse con el mejor. Es embarazosamente paralelo.
2. **(2 min) PRAM CREW:** muestra el Algoritmo 2. Señala pardo, la **lectura concurrente** (no “broadcast”), la reducción arg-máx. **Justifica la memoria compartida.**
3. **(2 min) Métricas:** W , T_∞ , T_p (con el $\log_2 p$), speedup, eficiencia, FLOPs. Señala la fórmula con el término de reducción.
4. **(3 min) Resultados:** dos plataformas (Mac ~ 5 , **Khipu ~ 10.6**), la comparación valida la escalabilidad. Eficiencia (responde “ $\mathcal{E}$ constante?” $\rightarrow$ strong cae, weak se mantiene), validación teórica ($R^2 = 0,989$), GFLOP/s.
5. **(1 min) Mejoras propuestas.**
6. **(1 min) Conclusiones.**

56. Preguntas que hará el profe (con respuestas)

P: ¿Por qué el speedup no llega a los núcleos totales?

R: Dos causas: (1) **hyper-threading** — en el Xeon, 32 núcleos lógicos ≈ 16 físicos, así que el speedup real máximo está entre 10 y 16 (medimos ~ 10.6); (2) **contención de memoria** — el

MLP es memory-bound y a alto p los procesos compiten. La reducción arg-máx es $< 1,5\%$, así que NO es la comunicación. La prueba: en la Mac (10 núcleos, sin HT) satura en ~ 5 , y en Khipu (32) llega a ~ 10.6 — el speedup crece con el hardware.

P: ¿Por qué corrieron en dos máquinas?

R: Para validar la escalabilidad en hardware distinto. Mismo código, mismo dataset, dos plataformas de memoria compartida. Mostrar que el speedup se duplica al pasar de 10 a 32 núcleos valida experimentalmente que el algoritmo escala.

P: ¿Por qué no usaron MPI?

R: El problema es embarazosamente paralelo y los datos son compartidos de solo lectura. En MPI habría que copiar el dataset a cada proceso (memoria $\times p$). En memoria compartida cada proceso hereda los datos por fork sin copiar. MPI no aportaría nada y sumaría overhead.

P: ¿Es realmente OpenMP si usan Python?

R: El MODELO es OpenMP (memoria compartida, fork-join). La IMPLEMENTACIÓN es `multiprocessing` porque el GIL impide hilos reales. Conceptualmente es lo mismo.

P: ¿Cómo verificaron que el paralelo da lo mismo que el secuencial?

R: El código es determinista (semillas fijas). Comparamos las 32 accuracies y el modelo ganador entre sec y par: **idénticos**.

P: ¿Qué es el “worker crítico” que miden?

R: El tiempo de la región paralela = el worker más lento. Medir la suma sería medir el trabajo W , no el tiempo T_p .

P: ¿Por qué no midieron la lectura de datos?

R: Por la observación del profesor: no incluir I/O en la medición de tiempos. Medimos solo la región paralelizable + la reducción.

P: ¿De dónde sale el $\log_2 p$ en T_p ?

R: De la reducción arg-máx en árbol binario: con p resultados locales, hace $\log_2 p$ rondas para encontrar el máximo global.

P: ¿El algoritmo es escalable?

R: Sí. Strong scaling: lineal hasta $p = 4$. Weak scaling: $E \approx 1$. El límite es el ancho de banda de memoria, no el algoritmo.

P: ¿El algoritmo es WT-óptimo?

R: Sí. $C_p = T_p \cdot p = O(W)$ porque $p \leq p_s = W/T_\infty$.

P: ¿Cómo calcularon los FLOPs?

R: A partir de la arquitectura: $4 \cdot E \cdot h \cdot (d + c) \cdot n$ por MLP. No usamos hw-counters; contamos las operaciones del costo conocido del MLP. $\text{GFLOP/s} = \text{FLOPs totales}/T_p$.

57. Trampas que caza el profe

- Decir “**broadcast**” en memoria compartida $\rightarrow$ di “**lectura concurrente CREW**”.
- Confundir tiempo paralelo con **suma** de tiempos $\rightarrow$ es el **máximo** (worker crítico).
- Decir que $E(p)$ siempre es 1 $\rightarrow$ cae con p (solo se mantiene en weak scaling).
- Decir “usamos OpenMP” sin aclarar $\rightarrow$ “modelo OpenMP, implementado con `multiprocessing` por el GIL”.

PARTE X — Chuleta final

58. Diccionario rápido

Término	Significado
T_s	tiempo secuencial (con 1 procesador)
T_p	tiempo paralelo (con p procesadores)
$S(p) = T_s/T_p$	speedup (cuántas veces más rápido)
$E(p) = S(p)/p$	eficiencia (aprovechamiento, ≤ 1)
W	work = trabajo total = $O(T_s)$
T_∞	span = camino crítico (tiempo con ∞ procesadores)
$C_p = T_p \cdot p$	costo
PRAM	modelo teórico de máquina paralela
CREW	lectura Concurrente, Escritura Exclusiva
pardo	bucle en paralelo
Brent	$T_p \leq W/p + T_\infty$
Amdahl	techo de speedup por fracción secuencial
Gustafson	speedup casi lineal (weak scaling)
GIL	candado de Python que impide hilos paralelos
fork	crear proceso hijo que hereda memoria
MLP	perceptrón multicapa (red neuronal)
η	learning rate
E	épocas
accuracy	fracción de aciertos
p_s	número de semillas (32)

59. Las fórmulas que debes saber

$$S(p) = \frac{T_s}{T_p} \quad E(p) = \frac{S(p)}{p} \quad T_p \leq \frac{W}{p} + T_\infty \quad T_p \approx a \frac{p_s}{p} n + b \log_2 p$$

$$\text{FLOPs por MLP} = 4Eh(d+c)n \quad \text{GFLOP/s} = \frac{\text{FLOPs totales}}{T_p \cdot 10^9}$$

60. Los números del proyecto (memorízalos)

- $p_s = 32$ semillas, $d = 32$, $h = 128$, $c = 10$, $E = 100$ épocas.
- Dos plataformas: MacBook M5 (10 núcleos) y **cluster Khipu n003** (Xeon Gold 6130, 32 núcleos lógicos ≈ 16 físicos con HT).
- Speedup techo: **~ 5 (Mac)**, **~ 10.6 (Khipu)** — crece con el hardware.
- Reducción arg-máx $< 1,5\%$ del tiempo (embarassingly parallel).
- Ajuste teórico (Khipu): $a = 2,42 \times 10^{-4}$ s/muestra, $b = 2,18$ s, $R^2 = 0,989$.
- GFLOP/s pico: **~ 53 (Mac)**, **~ 76 (Khipu)**.
- Por qué no llega a p : hyper-threading (32 lógicos $\neq$ 32 físicos) + contención de memoria (MLP memory-bound).

Regla de oro

Si el profe pregunta algo que no medimos, di “no lo medimos, pero...” y da la respuesta cualitativa correcta. **Nunca inventes números.**